

Lean Formalization of the Parameter Classification for Conditional Entropies

Roberto Rubboli, Erkkä Haapasalo, and Marco Tomamichel

This note records the Lean formalization of the necessary-and-sufficient parameter classification for the concrete conditional-entropy candidates of Refs. [1–3]. It gives a compact theorem-facing proof map, a statement-by-statement correspondence table, and the precise trust and scope boundary. The presentation is deliberately theorem-facing: it separates the public conclusions, their paper-level ingredients, and the proved reusable infrastructure.

1. LEAN CORRESPONDENCE

The formal development follows the mathematical proof in the concise and full-detail companion papers. It covers the endpoint-aware Rényi parameter, real-power and entropy differentiation, signed integration, common discretization, the two-block, three-block, and Shannon localizers, exact tropical stationarity, all sufficiency and necessity branches, and the original finite semiring and conditionally mixing channel lift. The source is available at <https://github.com/roberto-rubboli/A-complete-characterization-of-conditional-entropies> under the release tag `four-document-release-2026-08-25`.

Throughout this note, every numbered reference to a lemma, proposition, or remark is the number in the full-details Sections 4 and 5 paper, Ref. [3]. The concise companion, Ref. [2], uses the identical statement order and numbering, so the same label locates the corresponding concise statement.

There are two public conclusions. The declaration `mainClassification` says that, for every probability measure on the compactified Rényi parameter space, monotonicity of each concrete candidate is equivalent to its exact admissibility condition. Its only explicit argument is the measure being classified; it has no auxiliary hypothesis. The closed declaration `allConditionalEntropyAxioms` proves nonnegativity, zero-embedding invariance, tensor additivity, fair-bit normalization, and embedded channel monotonicity for every admissible candidate. Neither theorem assumes a project-specific axiom.

The repository contains three complementary diagrams at different levels of detail. Figure 1 is the *theorem-facing proof map*: it connects public conclusions to numbered manuscript results and reusable proof ingredients. It is not a module-import graph. The standalone files `paper/dependency-graph.dot`, `paper/dependency-graph.svg`, and `paper/dependency-graph.png` are three formats of one detailed *Lean implementation dependency graph*; their arrows run from prerequisite to dependent. The main-paper Lean appendix contains a third, coarser *condensed two-branch architecture overview* using that same source-level arrow direction.

In Figure 1, arrows deliberately point in the reverse direction, from a conclusion to ingredients used in its proof. Pale green boxes are the two public conclusions, white boxes are paper-facing intermediate results, pale blue boxes are reusable infrastructure that is itself proved in Lean, and the dashed gray box is an upstream representation theorem cited from Section 3 of Ref. [1]. That gray result is needed only for the statement that every abstract derivation has the displayed barycentric form; it is not an axiom or hypothesis of either green Lean declaration.

Table I follows, statement by statement, the numbering of the full-details Sections 4 and 5 paper, Ref. [3]. The concise companion uses the same numbers. All Lean names are in the namespace `ConditionalEntropy`. A split API means that one manuscript statement is intentionally proved by several smaller declarations. The last row is worded differently because Proposition 2.10 combines the formalized parameter restriction with the upstream representation theorem identified above.

The table records the human-facing correspondence. The repository also keeps the complete

TABLE I. Statement-by-statement correspondence. All lemma, proposition, and remark numbers are those of the full-details Sections 4 and 5 paper, Ref. [3]; the concise companion uses the same numbering.

Manuscript statement	Lean declaration	Status
Lemma 1.1: power-mean curvature	<code>finiteHolder; powerCurvature; parameterPowerCurvature</code>	Formalized
Lemma 1.2: monomial curvature	<code>affineMonomialCurvaturePackage</code>	Formalized
Lemma 1.3: finite-support sufficiency	<code>finiteSufficiency</code>	Formalized
Proposition 1.4: general temperate sufficiency	<code>generalSufficiency; finiteTSufficiency</code>	Formalized (split API)
Proposition 1.5: negative-tropical sufficiency	<code>negativeTropicalSufficiency</code>	Formalized
Proposition 1.6: positive-tropical sufficiency	<code>positiveTropicalComponent; positiveTropicalSufficiency</code>	Formalized (split API)

Table I (continued; statement numbers are from the full-details Sections 4 and 5 paper).

Manuscript statement	Lean declaration	Status
Remark 1.7: exact tropical endpoint families	<code>Tropical fields of mainClassification</code>	Formalized (split API)
Proposition 1.8: derivation sufficiency	<code>derivationSufficiency</code>	Formalized
Proposition 2.1: positive necessity	<code>positiveNecessity; positiveTemperateProbabilityNecessity</code>	Formalized (split API)
Proposition 2.2: negative Shannon atom with upper support	<code>negativeShannonObstruction</code>	Formalized
Proposition 2.3: truncated exceptional-moment bound	<code>truncatedMoment; exceptionalUpperMeasurePackage; MLower_le_of_truncated_difference</code>	Formalized (split API)

Table I (continued; statement numbers are from the full-details Sections 4 and 5 paper).

Manuscript statement	Lean declaration	Status
Remark 2.4: homogeneity and channel lift	<code>columnPhi_posHomOne; columnDeriv_posHomOne; conditionalSemiringOfJointProb_le; embeddingleift</code>	Formalized (split API)
Remark 2.5: compensated two-column witness	<code>normalizedTwoColumn; normalizedTwoColumn_column_zero; normalizedTwoColumn_column_one; mergeTwoChannel_output_column; sum_normalizedTwoColumn</code>	Formalized (split API)
Proposition 2.6: at most one upper support point	<code>oneUpperPoint; negativeTemperateNecessity_of_atom_zero</code>	Formalized (split API)

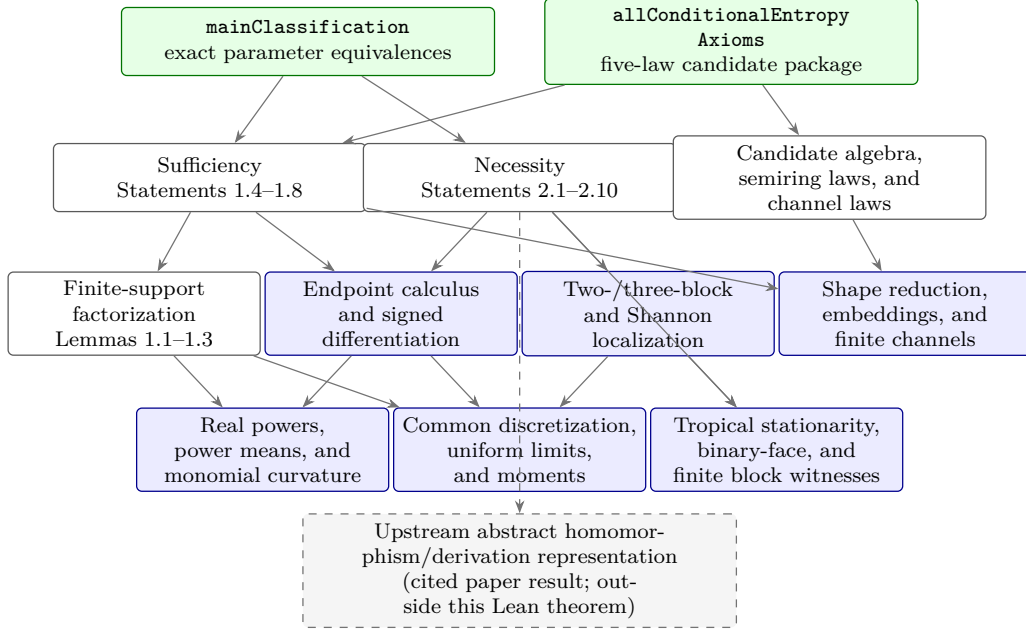


FIG. 1. Theorem-facing proof map for the four-document release. Every displayed statement number is the number in the full-details Sections 4 and 5 paper, Ref. [3]; the concise companion uses the same numbering. Arrows point from a result to ingredients used in its proof, the reverse of the source-dependency convention in the module-group graph. Color records proof role, not confidence: green means public conclusion, blue means internally proved infrastructure, white means a paper-facing result, and dashed gray marks the explicitly excluded upstream representation theorem. The dashed connector records manuscript context, not a Lean axiom or dependency.

Table I (continued; statement numbers are from the full-details Sections 4 and 5 paper).

Manuscript statement	Lean declaration	Status
Proposition 2.7: negative-tropical necessity	<code>negativeTropicalUpperAtomZero</code> ; <code>twoUpperSupportPointsTropicalObstruction</code> ; <code>tropicalTruncatedMoment_of_exceptional</code> ; <code>negativeTropicalNecessity</code>	Formalized (split API)
Proposition 2.8: positive-tropical necessity	<code>sameSupportDecomposition</code> ; <code>probabilityEqDiracZero</code> ; <code>positiveTropicalNecessity</code>	Formalized (split API)
Proposition 2.9: derivation necessity	<code>derivationNecessity</code>	Formalized
Proposition 2.10: represented derivations	<code>derivationSufficiency</code> ; <code>derivationNecessity</code> ; <code>derivation field of mainClassification</code>	Range proved; representation cited

implementation ledger with 152 source-ordered rows, together with the declaration index and editable Lean implementation dependency graph. Those longer audits are kept out of this note so that the main proof route remains readable.

A. Trust boundary and reproducibility

The project is pinned to Lean 4.32.2 and Mathlib 4.32.2. The release source contains no `sorry`, `admit`, project-defined `axiom`, or project-defined `opaque` declaration. The final declarations contain

no unproved project hypothesis. Lean’s standard logical foundations, including classical choice, propositional extensionality, and quotient soundness, remain part of the ordinary trusted base.

The single command

```
powershell -File scripts/check.ps1
```

runs the forbidden-token scan, a warning-as-error build of the integrated **CompleteCharacterization** target, the axiom audit, and the exact correspondence audit. The formalization proves the classification and law package for the four concrete candidate families. The upstream representation of every abstract homomorphism or derivation, the later barycentric classification of every abstract conditional entropy, and the large-sample, catalytic, and application results remain results of Ref. [1], not hidden assumptions of the Lean terminal theorems.

-
- [1] R. Rubboli, E. Haapasalo, and M. Tomamichel. “A Complete Characterisation of Conditional Entropies”. Manuscript, (2026). Available online: <https://arxiv.org/abs/2601.23213>. The equation and theorem numbers cited by the companion papers refer to the Ultra Lean revision.
 - [2] R. Rubboli, E. Haapasalo, and M. Tomamichel. “Sufficient and Necessary Conditions for the Parameters of Conditional Entropies”. Companion note, (2026). Concise presentation of the exact parameter classification.
 - [3] R. Rubboli, E. Haapasalo, and M. Tomamichel. “Sufficient and Necessary Conditions for the Parameters of Conditional Entropies: Complete Proofs”. Companion proof paper, (2026). Full analytic proofs of the parameter classification.